The ImGee UX Design Manifesto

Ergonomics, Cognitive Affordances, and Human Factors for Simulation User Interfaces

---

Authors & Architecture: Steve Goldberg & the Antigravity Engineering Team

Custodian: Managed for the ImGee platform (ImgWindow / ImGee / LuiGee / PyGee)

Status: Canonical Platform Standard

Last Updated: November 2026

---

1. The Founding Premise: Cockpit Cognitive Reality

Flight simulation tools, glass-cockpit avionics, and mission utilities operate in a fundamentally different cognitive domain than web applications or office productivity software:

- **High Cognitive Load:** The pilot is actively flying the aircraft, trimming pitch, monitoring engine instruments, communicating with ATC, and maintaining situational awareness.
- **Peripheral & Fast-Action Modality:** Interactions with plugins and panels happen in quick, fleeting bursts—often using one hand on a mouse while the other is on a flight stick or throttle.
- **The Golden Human Factors Rule:**

> **Protect Expert Mode. Never force the user into Thinking Mode.**

>

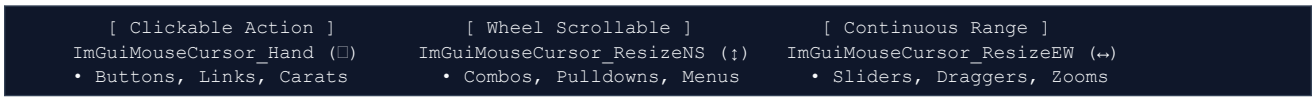
> In **Expert Mode**, the user operates entirely through spatial intuition, peripheral visual cues, and muscle memory. In **Thinking Mode**, the user is forced to pause, abandon aircraft control, squint at the UI, and diagnose why an element failed to respond to an input gesture. Every UI failure that triggers Thinking Mode is an architectural defect.

Stock Dear ImGui is engineered for game-engine debuggers, where developers sit comfortably at desks inspecting memory trees with standard mice. ImGee re-engineers these foundations into an opinionated, tactile design language engineered specifically for simulation cockpits.

---

2. Cursor Affordances & The "Zero-Inert" Law

A mouse pointer is not merely an indicator of (X, Y) screen coordinates; it is the user's primary tactile feedback channel. In the absence of physical switch detents, the OS mouse cursor provides instant confirmation of what a widget is prepared to do.



The Zero-Inert Law

> **An interactive, hot, or clickable widget must NEVER leave the mouse cursor on the stock `Arrow` (`ImGuiMouseCursor_Arrow`).**

- Leaving an interactive element on `Arrow` tells the user's subconscious that the control is passive text or dead space.
- Every interactive ImGee widget must declare its affordance explicitly:

1. `ImGuiMouseCursor_Hand` **(Point & Trigger):** Clickable pushbuttons, toggle rocker switches, hyperlinked text (`ButtonLink`, `TextLink`), and closed dropdown chevrons. Signals: *"Clicking this causes an action or toggles a menu."*
2. `ImGuiMouseCursor_ResizeNS` **(Vertical Cycle & Step):** Discrete multi-choice selectors, open dropdown option lists (`SelectPulldown`, `MouseWheelCombo`), and vertical value cyclers. Signals: *"Spinning the mouse-wheel will step through options."*
3. `ImGuiMouseCursor_ResizeEW` **(Continuous Horizontal Modulation):** Continuous range sliders (`SliderFloat`, `SliderInt`), drag controls, and zoom adjusters. Signals: *"Dragging horizontally or spinning the wheel smoothly modulates a continuous range."*
4. `ImGuiMouseCursor_TextInput` **(Text Entry):** Interactive text fields and scratchpads (`InputText`, `InputTextMultiline`).

---

3. The First-Gesture Discoverability Law

When an expert user encounters a new or unfamiliar dropdown or pulldown widget, they perform a **micro-gesture**—a single notch of the mouse wheel—to discover if the widget is interactive.

Natural List Order

> **Scrolling the wheel DOWN (negative delta) MUST ALWAYS advance +1 down the visual list (0 → 1 → 2).**

```
    [ Item 0 (Charcoal) ]   ▲ Wheel UP   (-1 delta)
    [ Item 1 (Rusty)    ]   ▼ Wheel DOWN (+1 delta)
    [ Item 2 (Dark)     ]
```

The Initial-Gesture False Negative

If directional mapping is inverted under the hood (e.g. scrolling down decrements index rather than incrementing), an insidious cognitive failure occurs:

1. The user sees `Item 0` selected at the top of the list.
2. The user spins the mouse-wheel **down** to move to `Item 1`.
3. Because the internal logic decrements, it tries to select `-1`, which immediately hits the boundary clamp at `0`.
4. **Nothing moves on the screen.**
5. The pilot's brain reaches an immediate, completely logical, yet false conclusion: *"This widget does not support the mouse-wheel."*

The pilot never considers spinning *up* from the top item. Discoverability is destroyed on gesture one. Directional mapping must strictly mirror visual layout geometry across all platforms and bindings.

---

4. The Monotonic vs. Categorical Dialectic (Clamping vs. Wrapping)

Not all enumerable lists are created equal. When enabling mouse-wheel navigation, widget authors and developers must classify data into one of two fundamental categories:

A. Monotonic / Scalar Ranges (`wrap = false`)

- **Characteristics:** The options represent quantitative magnitude, physical distance, severity, or safety-critical states.
- **Examples:**
- Altimeter minimums, target distances (`0 m .. 1000 m`).
- Audio volume levels, display brightness (`0% .. 100%`).
- System power modes (`Off`, `Standby`, `Armed`, `Active`).

- **The Rule: Clamping at boundaries is mandatory.** Wrapping around from `1000 m` straight to `0 m` (or `Max` to `Off`) is disorienting, counter-intuitive, and in aviation software, potentially hazardous.
- **Ergonomic Requirement:** When hitting a boundary, the control must stop cleanly without overshoot.

B. Categorical / Carousel Sets (`wrap = true`)

- **Characteristics:** The options are qualitative presets, unordered modes, or cyclic vantage points where no intrinsic numeric floor or ceiling exists.
- **Examples:**
- Theme palettes (`Charcoal` → `Rusty` → `Dark` → `Classic` → `Light`).
- Camera views (`Cockpit 3D` → `Chase 30m` → `Tower View`).
- Navigation display modes (`Rose`, `Arc`, `Map`, `Plan`).
- **The Rule: Circular wrapping provides friction-free fluidity.** Hitting a dead end on an arbitrary palette or camera list forces the user to reverse wheel direction unnecessarily. Wrapping turns the control into an infinite carousel.

The Developer API Contract

Because a reusable widget cannot deduce the semantics of caller-provided strings at compile time:

- All ImGee enumerable widgets must default to safe clamping (`wrap = false`).
- All ImGee enumerable widgets must expose an explicit `wrap` parameter across all overloads:

```cpp
ImGui::SelectPulldown("##CameraMode", &s_CamMode, items, count, tooltips, width, anchor, align, /*wrap=*/true)
ImGui::MouseWheelCombo("##ThemePicker", &s_Theme, items, count, -1, /*wrap=*/true, tooltips);
```

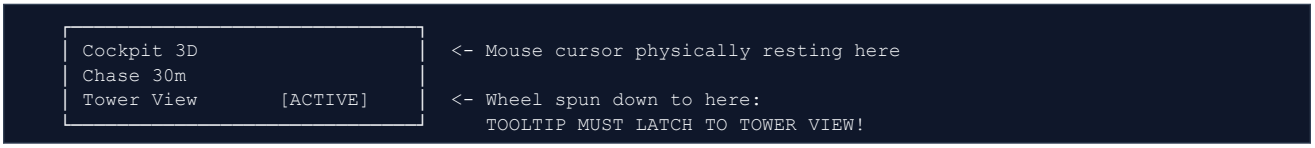
---

5. The "Lens UI" Principle: Modal Focus in Navigation

When a dropdown or pulldown menu is opened, the user's interaction modality is fluid. They may click an item with the mouse, or they may keep the mouse stationary and spin the wheel to preview options.

Spatial Hover vs. Navigational Selection Focus

- When the mouse is moving across items, the active focus is **spatial hover**: the tooltip displays information about the row physically beneath the mouse cursor.
- The instant the user spins the mouse-wheel, their mental focus shifts to **navigational selection focus**: the user is actively driving the selection index (`*current_item`), and expects to inspect that newly selected item.



The Latching Contract

1. When the mouse wheel steps `*current_item`, ImGee activates a **tooltip latch** (`s_WheelLatchPulldownID = id`).
2. While latched, tooltips target `items[*current_item]` dynamically, ignoring the stationary mouse pointer.
3. The user can rapidly spin the wheel through 10 items and read the context-sensitive tooltip for each successive selection in real time (the "Lens" effect).

The Hysteresis Threshold

- To prevent accidental unlatching caused by physical wheel vibrations (notched wheel detents often introduce a 1–2 px micro-jiggle on physical mice), the latch must ignore micro-movements.

- The latch is broken only when deliberate pointer motion is detected:

```
[TEXT]
  Δx > 6.0 px   or   Δy > 6.0 px
```

- Once deliberate movement occurs, the system immediately reverts to standard spatial hover. Clicking an item cleanly clears the latch.

---

6. Container Scroll Pass-Through vs. Control Capture

In dense configuration windows, dataref inspectors, and checklists, multiple pulldown controls sit inside long vertically scrollable windows.

```
  [ Settings Window: Scrolling Down... ]
    ├─ Radio Nav Source:    [ NAV 1 ▼ ]  <- Closed: WHEEL PASSES THROUGH! Window scrolls.
    ├─ Transponder Mode:    [ ALT   ▼ ]  <- Closed: WHEEL PASSES THROUGH! Window scrolls.
    └─ Camera Preset:       [ CHASE ▲ ]  <- Open: WHEEL CAPTURED! Options cycle.
```

- **When Closed:** 1-line pulldowns (`SelectPulldown`) **must never capture the mouse-wheel**. Wheel gestures must pass cleanly through to the parent scrollable container. This guarantees pilots can scroll through a 50-row checklist or dataref inspector without accidentally altering operational settings.
- **When Open:** Once the menu is actively opened by the user, the popup claims modal focus. Inside the open popup window, mouse-wheel gestures are captured to cycle selection options dynamically.

---

7. Tactile Feedback, Reversibility, and Non-Destructive Defaults

Exploration encourages mastery. If users fear that adjusting a setting will permanently corrupt their configuration, they hesitate to interact.

- **1-Click Restore to Default (`SliderWithReset`, `ComboWithReset`):**
- Every adjustable slider or tuning combo must provide an inline reset button (↺) that restores the factory baseline in a single click.
- Clicking reset a second time acts as instant **Undo**, restoring the user's previous custom value.
- **Destructive Guardrails:**
- Modal file pickers (`SaveFileDialog`) must provide built-in file overwrite confirmation dialogs.
- Dangerous actions (e.g. wiping calibration profiles) require two-stage confirmation or long-press actuation.

---

8. Typography, Markdown, and Visual Signaling

Color and formatting are not decorative; in aviation interfaces, they are semantic status indicators.

- **Contrast Integrity:** Text must never blend into its background. If a style tag omits a foreground color (e.g. `<style bg=#444444>`), the rendering engine must calculate luminance and apply high-contrast text automatically.
- **Semantic Tagging Hierarchy:**
- **Capsule Tags (`<pill>`, `<badge>`, `<backdrop>`):** Define physical rounded boundaries with semantic background fills. Used for operational status (`<pill color=#2ecc71>ACTIVE</pill>`) and hardware states.
- **Text Style Tags (`<style>`, `<col>`, `<cam>`):** Modulate typography (font, weight, color) without injecting container boxes.
- **Template Aliases (`<tip>`, `<hotkey>`):** Provide turnkey, standardized visual stamps for operational guidance and keybindings.
- **Font-Scale Invariance:** Window chrome, status bars, and title bars must maintain a stable 1.0x baseline scale (`SetUnscaledChrome`), ensuring operational controls and indicators remain rock-solid regardless of internal document

zoom.

---

9. Summary: The ImGee Checklist for Every Widget

Before any widget is approved for release into ImGee, LuiGee, or PyGee, verify the following:

- [ ] Does hovering the closed control display the appropriate non-arrow cursor (`Hand` for triggers, `ResizeNS` for combos)?
- [ ] Does mouse-wheel navigation advance down the list (+1) on scroll down (`natural_list_order = true`)?
- [ ] Is boundary clamping vs. circular wrapping explicitly chosen and exposed via a `wrap` parameter?
- [ ] Does the open dropdown support "Lens UI" tooltip latching during wheel navigation?
- [ ] Does the tooltip latch maintain a > 6 px hysteresis threshold against physical mouse wheel jiggle?
- [ ] Does the closed control pass wheel events through to parent scrollable containers?
- [ ] Do tooltips support full Markdown formatting without raw markdown character leaks?
- [ ] Does the widget provide 1-click restore or undo history if it controls tuning parameters?